

Air Quality Index Classification Using Machine Learning Algorithms

Thota Vivek (Leader)
S20230010244

B. Pardhuva
S20230010045

K. Hrishikesh
S20230010132

Chedideepu Jayanth
S20230010057

Baikadi Vignesh
S20230010038

*Department of Computer Science and Engineering
IIT Sricity*

Abstract—This term paper will focus on comparing the performance of selected classification algorithms for analyzing urban air quality. The primary goal is to explore how different machine learning approaches can be utilized to predict and group cities based on key air quality indicators such as PM2.5, PM10, CO, NO, SO, and O levels. Publicly available datasets, from Kaggle, will be used to ensure the availability of diverse and reliable environmental data. The project will implement four distinct algorithms — Logistic Regression, Support Vector Machine (SVM), Decision Tree and Naïve Bayes— to evaluate their effectiveness in modeling air quality patterns. Their performance will be evaluated using accuracy and macro/weighted F1 on a held-out evaluation set. Through this comparative approach, the study aims to determine the most suitable algorithm for understanding and predicting air quality trends across urban regions.

Index Terms—Logistic Regression, Support Vector Machine, Decision Tree, Naive Bayes, Gradient Descent, Quadratic Programming, Air Quality Index, AQI Classification, From-Scratch Implementation, Class Imbalance, Feature Importance, SMOTE

I. INTRODUCTION

Air quality has emerged as one of the most pressing environmental challenges of the 21st century, with significant implications for public health, climate change, and sustainable urban development. The Air Quality Index (AQI) serves as a standardized metric to communicate air quality levels to the public, categorizing air quality into six distinct classes: Good, Satisfactory, Moderate, Poor, Very Poor, and Severe.

The accurate prediction and classification of AQI categories is essential for:

- **Public health protection through early warning systems**
- **Environmental policy formulation and regulation**
- **Urban planning and infrastructure development**
- **Research on air pollution patterns and trends**

Traditional statistical methods for air quality prediction often struggle with the non-linear relationships, high-dimensional feature spaces, and class imbalance inherent in environmental datasets. Machine learning approaches offer promising alternatives by automatically learning complex patterns from data without explicit programming of rules.

This study presents a comprehensive evaluation of **four fundamental machine learning classifiers** for AQI category prediction:

- 1) **Logistic Regression** - A linear classifier with probabilistic outputs
- 2) **Support Vector Machine (SVM)** - A maximum-margin classifier with kernel methods
- 3) **Decision Tree** - A non-parametric tree-based classifier
- 4) **Naive Bayes** - A probabilistic classifier based on Bayes' theorem

Each classifier was implemented from scratch with state-of-the-art enhancements including regularization, optimization techniques, and memory-efficient processing. The models were evaluated on two distinct datasets to assess generalizability and performance consistency.

The primary objectives of this research are:

- To implement and compare four machine learning classifiers for AQI classification
- To evaluate model performance using comprehensive metrics (accuracy, precision, recall, F1-score)
- To analyze feature importance and model interpretability
- To assess scalability and computational efficiency
- To identify the most suitable classifier for air quality prediction tasks

II. DATASETS DESCRIPTION

This study leverages two complementary air quality datasets to evaluate the robustness of machine learning models across diverse AQI prediction scenarios: a multi-city dataset (`city_day_dataset.csv`) focused on various pollutants across India, and a city-specific dataset (`delhi_air_quality_dataset.csv`) centered on Delhi's air quality profiles.

A. *city_day_dataset.csv: Multi-City Air Quality Dataset*

The City Day Dataset is a comprehensive air quality dataset containing measurements from multiple cities over time. The dataset consists of 29,531 samples with 16 columns representing various air quality parameters and metadata.

Features:

- **City:** Categorical variable representing 26 different cities
- **Date:** Temporal information in datetime format
- **PM2.5:** Particulate matter with diameter less than 2.5 micrometers ($\mu\text{g}/\text{m}^3$)
- **PM10:** Particulate matter with diameter less than 10 micrometers ($\mu\text{g}/\text{m}^3$)
- **NO:** Nitrogen monoxide concentration ($\mu\text{g}/\text{m}^3$)
- **NO2:** Nitrogen dioxide concentration ($\mu\text{g}/\text{m}^3$)
- **NOx:** Nitrogen oxides concentration ($\mu\text{g}/\text{m}^3$)
- **NH3:** Ammonia concentration ($\mu\text{g}/\text{m}^3$)
- **CO:** Carbon monoxide concentration ($\mu\text{g}/\text{m}^3$)
- **SO2:** Sulfur dioxide concentration ($\mu\text{g}/\text{m}^3$)
- **O3:** Ozone concentration ($\mu\text{g}/\text{m}^3$)
- **Benzene:** Benzene concentration ($\mu\text{g}/\text{m}^3$)
- **Toluene:** Toluene concentration ($\mu\text{g}/\text{m}^3$)
- **Xylene:** Xylene concentration ($\mu\text{g}/\text{m}^3$)
- **AQI:** Numeric Air Quality Index value
- **AQI_Bucket:** Categorical target variable with 6 classes (Good, Satisfactory, Moderate, Poor, Very Poor, Severe)

Dataset Characteristics:

- **Total samples:** 29,531
- **Number of cities:** 26
- **Number of features used:** 13 (excluding AQI to prevent data leakage, and Date/City which are encoded)
- **Target classes:** 6 AQI categories
- **Class distribution (before SMOTE):** Moderate (13,510), Poor (8,224), Satisfactory (2,781), Very Poor (2,337), Good (1,341), Severe (1,338)
- **Missing values:** 88,488 total missing values across all columns (handled through median imputation)
- **Train-test split:** 70% training (20,671 samples), 30% testing (8,860 samples)
- **After SMOTE:** 57,030 training samples (balanced across all classes)

B. delhi_air_quality_dataset.csv: Delhi-Specific Air Quality Dataset

The Delhi Air Quality Dataset is a focused dataset containing air quality measurements specifically for Delhi, India. This dataset includes 1,461 samples with 12 columns.

Features:

- **Date:** Temporal information
- **Month:** Month of the year (1–12)
- **Year:** Year of measurement
- **Holidays_Count:** Number of holidays in the period
- **Days:** Day count
- **PM2.5:** Particulate matter 2.5 ($\mu\text{g}/\text{m}^3$)
- **PM10:** Particulate matter 10 ($\mu\text{g}/\text{m}^3$)
- **NO2:** Nitrogen dioxide concentration ($\mu\text{g}/\text{m}^3$)
- **SO2:** Sulfur dioxide concentration ($\mu\text{g}/\text{m}^3$)
- **CO:** Carbon monoxide concentration ($\mu\text{g}/\text{m}^3$)
- **Ozone:** Ozone concentration ($\mu\text{g}/\text{m}^3$)
- **AQI:** Numeric Air Quality Index value

Dataset Characteristics:

- **Total samples:** 1,461
- **Number of features used:** 8 (PM2.5, PM10, NO2, SO2, CO, Ozone, Month, Holidays_Count)
- **Target variable:** AQI_Bucket (derived from AQI values using standard thresholds)
- **Target classes:** 6 AQI categories
- **Class distribution (before SMOTE):** Moderate (463), Poor (384), Satisfactory (267), Very Poor (231), Severe (65), Good (51)
- **Train-test split:** 70% training (1,022 samples), 30% testing (439 samples)
- **After SMOTE:** Variable training samples depending on model (approximately 1,800–2,200 samples)

C. Dataset Summary

Both datasets share common air quality parameters including **PM2.5, PM10, NO2, SO2, CO, and Ozone**, which are standard indicators of air pollution. The City Day Dataset provides a **broader geographical scope with multiple cities**, while the Delhi dataset offers focused temporal analysis for a single location.

Key similarities:

- Both datasets contain standard air quality pollutants
- Both use the same AQI_Bucket classification system (6 categories)
- Both require similar preprocessing steps (missing value handling, encoding, scaling)
- Both exhibit class imbalance requiring SMOTE application

Key differences:

- **City Day Dataset:** Multi-city, larger sample size (29,531 vs 1,461), includes additional pollutants (NO, NOx, NH3, Benzene, Toluene, Xylene)
- **Delhi Dataset:** Single city focus, includes temporal features (Month, Year, Holidays_Count), smaller feature set

Data quality issues addressed:

- **Missing values:** Handled through median imputation for numeric columns
- **Class imbalance:** Addressed using SMOTE (Synthetic Minority Oversampling Technique)
- **Categorical encoding:** City and AQI_Bucket encoded using LabelEncoder
- **Feature scaling:** StandardScaler applied for models requiring normalized features
- **Memory optimization:** float32 data types used to reduce memory footprint

III. METHODOLOGY

This section details the comprehensive preprocessing pipeline and the from-scratch implementation of four machine learning classifiers: **Logistic Regression, Support Vector Machine (SVM), Decision Tree, and Naive Bayes**. All models were optimized for memory efficiency and performance on air quality datasets.

TABLE I: Comparative Summary of Datasets

Characteristic	city_day.csv	delhi_air_quality.csv
Dataset Type	Multi-City	City-Specific
Number of Records	29,531	1,461
Original Features	16	12
Final Features	13	8
Numerical Features	13	9
Categorical Features	3	3
Missing Values	Yes (88,488)	Yes
Class Distribution	Imbalanced	Highly Imbalanced
Feature Scaling	Standardization	Standardization
Train/Test Split	70/30	70/30
SMOTE Applied	Yes	Yes

A. Data Preprocessing

The preprocessing pipeline was designed to handle the unique characteristics of air quality datasets while maintaining data integrity and maximizing model performance.

1) Missing Value Handling:

- **Numeric columns:** Missing values filled with median of respective columns
- **Categorical columns:** AQI_Bucket missing values filled with 'Moderate' (most common class)
- **Rationale:** Median imputation preserves distribution better than mean for skewed data and avoids information loss from row deletion

2) Feature Engineering:

- Temporal features extracted from Date column: Year, Month, Day, DayOfWeek, DayOfYear, Quarter
- These features capture seasonal patterns and temporal trends in air quality
- Date and City columns removed after encoding to prevent direct use

3) Label Encoding:

- City column: Encoded to numeric values (26 unique cities in City Day Dataset)
- AQI_Bucket: Encoded to numeric labels (0–5 for 6 classes)
- LabelEncoder used for consistent mapping between original and encoded values

4) Feature Selection:

- AQI column explicitly excluded to prevent data leakage (target variable)
- All pollutant concentrations and derived features included
- **Feature columns (City Day Dataset):** City, PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene, Xylene
- **Feature columns (Delhi Dataset):** PM2.5, PM10, NO2, SO2, CO, Ozone, Month, Holidays_Count

5) Feature Scaling:

- StandardScaler applied: $(X - \mu)/\sigma$

- Essential for distance-based algorithms (SVM, Logistic Regression)
- Applied to both training and test sets (fit on training, transform on test)

6) Class Imbalance Handling:

- SMOTE (Synthetic Minority Oversampling Technique) applied to training data
- Generates synthetic samples for minority classes using k-nearest neighbors ($k = 5$)
- Results in balanced class distribution in training set
- Applied only to training data to prevent data leakage

7) Train-Test Split:

- **City Day Dataset:** 70% training (20,671 samples), 30% testing (8,860 samples), `random_state=0`
- **Delhi Dataset:** 70% training (1,022 samples), 30% testing (439 samples), `random_state=42`
- Stratified splitting considered but not always feasible due to class imbalance

8) Memory Optimizations:

- float32 data types used instead of float64 (50% memory reduction)
- Sample limits applied: `MAX_TRAIN_SAMPLES = 10,000--15,000` depending on model
- `MAX_SAMPLES_PER_CLASS = 3,000--5,000` for SMOTE to prevent excessive memory usage

B. Classifier 1: Logistic Regression

The Logistic Regression model uses Cross-Entropy Loss with L2 regularization and is optimized using Mini-Batch Gradient Descent with Momentum and Adaptive Learning Rate.

Cost Function:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] + \frac{\lambda}{2n} \|W\|^2 \quad (1)$$

Where:

- n : number of samples
- y_i : true class label (one-hot encoded for multiclass)
- $\hat{y}_i$: predicted probability from softmax
- λ : L2 regularization parameter ($\lambda_{reg} = 0.01$)
- W : weight matrix

For multiclass classification, the softmax function is used:

$$\sigma(z)_j = \frac{\exp(z_j)}{\sum_k \exp(z_k)} \quad (2)$$

The cross-entropy loss for multiclass becomes:

$$L = -\frac{1}{n} \sum_i \sum_j y_{ij} \log(\hat{y}_{ij}) + \frac{\lambda}{2n} \|W\|^2 \quad (3)$$

Gradient Descent:

1) Mini-Batch Processing:

- Batch size: 128 samples per batch
- Shuffles data at each epoch for better convergence

- Processes data in batches to balance convergence speed and computational efficiency
- 2) **Momentum Optimization:**
- Momentum coefficient: 0.9
 - Velocity update: $v_t = \beta \cdot v_{t-1} - \alpha \cdot \nabla L$
 - Weight update: $W_t = W_{t-1} + v_t$
 - Helps accelerate convergence and escape local minima
- 3) **Adaptive Learning Rate:**
- Initial learning rate: 0.01
 - Learning rate schedule:
 - Epochs 0–500: $lr = 0.01$
 - Epochs 500–1000: $lr = 0.005$ (50% reduction)
 - Epochs 1000–2000: $lr = 0.003$ (30% of original)
 - Epochs 2000–2500: $lr = 0.001$ (10% of original)
 - Epochs 2500+: $lr = 0.0005$ (5% of original)
 - Gradual decay prevents overshooting optimal solution
- 4) **Early Stopping:**
- Patience: 100 epochs
 - Monitors training loss
 - Stops if no improvement for patience epochs
 - Saves best model weights
- 5) **Weight Initialization:**
- Xavier/Glorot initialization: $\frac{W}{\text{Uniform}(-\sqrt{6/(n_{in} + n_{out})}, \sqrt{6/(n_{in} + n_{out})})} \sim$
 - Ensures appropriate initial weight scale for stable training

Implementation for `city_day_dataset.csv`:

- Training samples:** 57,030 (after SMOTE)
- Test samples:** 8,860
- Features:** 13
- Classes:** 6
- Hyperparameters:** $lr = 0.01$, $\lambda_{reg} = 0.01$, $batch_size = 128$, $momentum = 0.9$, $epochs = 3000$
- Results:**
 - Train Set:** Accuracy=0.7548, Precision=0.7554, Recall=0.7548, F1-Score=0.7536
 - Test Set:** Accuracy=0.7116, Precision=0.7533, Recall=0.7116, F1-Score=0.7217
 - Per-class performance (Test):**
 - * Good: Precision=0.3559, Recall=0.8677, F1=0.5047 (431 samples)
 - * Moderate: Precision=0.8499, Recall=0.7506, F1=0.7971 (4,005 samples)
 - * Poor: Precision=0.5154, Recall=0.7048, F1=0.5954 (830 samples)
 - * Satisfactory: Precision=0.7604, Recall=0.5993, F1=0.6703 (2,463 samples)
 - * Severe: Precision=0.7430, Recall=0.8171, F1=0.7783 (421 samples)
 - * Very Poor: Precision=0.7094, Recall=0.7324, F1=0.7207 (710 samples)

Implementation for `delhi_air_quality_dataset.csv`:

- Training samples:** 1,842 (after SMOTE)

- Test samples:** 439
- Features:** 8
- Classes:** 6
- Hyperparameters:** Same as above
- Results:**
 - Train Set:** Accuracy=0.7796, Precision=0.7841, Recall=0.7796, F1-Score=0.7763
 - Test Set:** Accuracy=0.6469, Precision=0.6800, Recall=0.6469, F1-Score=0.6551
 - Per-class performance (Test):**
 - * Good: Precision=0.3250, Recall=0.8667, F1=0.4727 (15 samples)
 - * Moderate: Precision=0.8346, Recall=0.6795, F1=0.7491 (156 samples)
 - * Poor: Precision=0.6475, Recall=0.6752, F1=0.6611 (117 samples)
 - * Satisfactory: Precision=0.6515, Recall=0.5972, F1=0.6232 (72 samples)
 - * Severe: Precision=0.4815, Recall=0.7222, F1=0.5778 (18 samples)
 - * Very Poor: Precision=0.5263, Recall=0.4918, F1=0.5085 (61 samples)

C. Classifier 2: Support Vector Machine

Soft-Margin SVM: The SVM implementation uses Soft-Margin formulation with CVXOPT quadratic programming solver and One-vs-One multiclass strategy.

Mathematical Formulation:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \xi_i \quad (4)$$

Subject to: $y_i(w^T x_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$
Where:

- w : weight vector
- b : bias term
- C : regularization parameter ($C = 0.1$ in our implementation)
- ξ_i : slack variables allowing margin violations
- y_i : class labels (-1 or $+1$ for binary classification)

Dual formulation (used in implementation):

$$\max_{\alpha} \sum \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad (5)$$

Subject to: $0 \leq \alpha_i \leq C$, $\sum \alpha_i y_i = 0$

Where:

- α_i : Lagrange multipliers (dual coefficients)
- $K(x_i, x_j)$: kernel function
- C : regularization parameter ($C = 0.1$)

Implementation Details:

1) Quadratic Programming Solver:

- CVXOPT library used for solving the QP problem
- Maximum iterations: 200
- Tolerance: $feastol = 1e-6$, $abstol = 1e-6$, $reltol = 1e-6$

2) Kernel Functions:

- **Linear Kernel:** $K(x_i, x_j) = x_i^T \cdot x_j$ (used for large datasets)
- **RBF Kernel:** $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$, $\gamma = 1/n_{features}$
- **Polynomial Kernel:** $K(x_i, x_j) = (\gamma \cdot x_i^T \cdot x_j + coef0)^{degree}$

3) One-vs-One Multiclass Strategy:

- For 6 classes: 15 binary classifiers trained
- Prediction: Voting mechanism (class with most votes wins)

4) Memory Optimizations:

- float32 data types
- Linear kernel for datasets >5,000 samples
- Sequential processing (n_jobs=1)
- Sample limits: MAX_TRAIN_SAMPLES = 15,000

Results:

- **City Day Dataset:**
 - Training samples: 8,000 (limited)
 - Kernel: Linear
 - $C = 0.1$
 - Accuracy: 0.7077 (70.77%)
 - Precision: 0.7394 (73.94%)
 - Recall: 0.7077 (70.77%)
 - F1-Score: 0.7063 (70.63%)
- **Delhi Dataset:**
 - Accuracy: 0.5626 (56.26%)
 - Precision: 0.6223 (62.23%)
 - Recall: 0.5626 (56.26%)
 - F1-Score: 0.5757 (57.57%)

D. Classifier 3: Decision Tree

Description: The Decision Tree classifier builds a tree-like model of decisions based on feature values. Each internal node represents a test on a feature, each branch represents the outcome of the test, and each leaf node represents a class label.

Algorithm:

1) Tree Construction (Recursive):

- Start with root node containing all training samples
- For each node:
 - a) Check stopping criteria (max_depth, min_samples_split, pure node)
 - b) If stopping criteria met: create leaf node with majority class
 - c) Otherwise: find best split (feature + threshold) using Gini impurity
 - d) Split data into left and right child nodes
 - e) Recursively build left and right subtrees

Splitting Criterion - Gini Impurity:

$$\text{Gini}(D) = 1 - \sum_{i=1}^C p_i^2 \quad (6)$$

Information Gain:

$$\text{Gain}(D, A) = \text{Gini}(D) - \sum_v \frac{|D_v|}{|D|} \text{Gini}(D_v) \quad (7)$$

Stopping Criteria:

- Maximum depth reached (max_depth = 10)
- Minimum samples for split not met (min_samples_split = 5)
- Node is pure or gain below threshold (min_gain = 1e-7)

Hyperparameters:

- max_depth: 10
- min_samples_split: 5
- min_gain: 1e-7

Feature Importance:

- Calculated as: $\sum (\text{gain} \times n_{\text{samples}})$ for each feature
- Normalized to sum to 1.0

Results:

- **City Day Dataset:**
 - Training samples: 10,000 (limited for memory)
 - Test samples: 8,860
 - Features: 13
 - Accuracy: 0.7764 (77.64%)
 - Precision: 0.7821 (78.21%)
 - Recall: 0.7764 (77.64%)
 - F1-Score: 0.7784 (77.84%)
 - **Per-class metrics:**
 - * Good: Precision=0.5680, Recall=0.7328, F1=0.6400
 - * Moderate: Precision=0.8550, Recall=0.8176, F1=0.8359
 - * Poor: Precision=0.6050, Recall=0.6576, F1=0.6302
 - * Satisfactory: Precision=0.7761, Recall=0.7761, F1=0.7761
 - * Severe: Precision=0.7232, Recall=0.7953, F1=0.7575
 - * Very Poor: Precision=0.7401, Recall=0.6927, F1=0.7157
 - **Top features:** PM2.5 (0.5053), CO (0.2018), PM10 (0.1505)
- **Delhi Dataset:**
 - Training samples: ~2,004 (after SMOTE with limits)
 - Test samples: 439
 - Features: 10
 - Accuracy: 0.7016 (70.16%)
 - Precision: 0.7178 (71.78%)
 - Recall: 0.7016 (70.16%)
 - F1-Score: 0.7039 (70.39%)
 - **Per-class metrics:**
 - * Good: Precision=0.3704, Recall=0.6667, F1=0.4762
 - * Moderate: Precision=0.7869, Recall=0.7442, F1=0.7649
 - * Poor: Precision=0.7197, Recall=0.7983, F1=0.7570

- * Satisfactory: Precision=0.7532, Recall=0.6591, F1=0.7030
- * Severe: Precision=0.4400, Recall=0.7333, F1=0.5500
- * Very Poor: Precision=0.6786, Recall=0.5205, F1=0.5891

E. Classifier 4: Naive Bayes

The Naive Bayes implementation uses Gaussian Kernel Density Estimation (KDE) with bandwidth $h = 0.1$.

Mathematical Formulation: For each class c and feature f , the probability density is estimated using:

$$P(x_f | \text{class} = c) = \frac{1}{n} \sum_{i=1}^n K_h(x_f - x_i) \quad (8)$$

Where:

- $K_h(u) = \frac{1}{h} \cdot K\left(\frac{u}{h}\right)$ is the kernel function
- $K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right)$ is the Gaussian kernel
- h : bandwidth parameter (0.1 in our implementation)
- n : number of training samples in class c

The Naive Bayes assumption:

$$P(x | \text{class} = c) = \prod_f P(x_f | \text{class} = c) \quad (9)$$

Posterior probability:

$$P(\text{class} = c | x) = \frac{P(x | \text{class} = c) \cdot P(\text{class} = c)}{P(x)} \quad (10)$$

Where:

- $P(\text{class} = c)$: prior probability = $\frac{\text{number of samples in class } c}{\text{total samples}}$
- $P(x)$: normalization constant

Results:

- **City Day Dataset:**
 - Training samples: Variable (after SMOTE)
 - Test samples: 4,970
 - Features: Variable (after preprocessing)
 - Accuracy: 0.68 (68.00%)
 - Precision: ~ 0.68 (weighted average)
 - Recall: ~ 0.68 (weighted average)
 - F1-Score: ~ 0.68 (weighted average)
- **Delhi Dataset:**
 - Training samples: 2,220 (after SMOTE)
 - Test samples: 293
 - Features: 6
 - Accuracy: 0.6894 (68.94%)
 - Precision: 0.7013 (70.13%)
 - Recall: 0.6894 (68.94%)
 - F1-Score: 0.6927 (69.27%)

F. Evaluation Metrics

The following metrics were used to evaluate all classifiers:

- 1) **Accuracy:** $(TP + TN) / (TP + TN + FP + FN)$
- 2) **Precision (Weighted):** $TP / (TP + FP)$
- 3) **Recall (Weighted):** $TP / (TP + FN)$

- 4) **F1-Score (Weighted):** $2 \times (P \times R) / (P + R)$
- 5) **Per-Class Metrics:** Precision, Recall, F1-Score for each AQI category
- 6) **Confusion Matrix:** Shows actual vs predicted class distribution
- 7) **Classification Report:** Comprehensive per-class metrics with support

IV. RESULTS AND ANALYSIS

This section presents the performance evaluation of four machine learning classifiers—Logistic Regression, Support Vector Machine (SVM), Decision Tree, and Naive Bayes—implemented from scratch on two air quality datasets. All models were evaluated using standard classification metrics: accuracy, precision, recall, and F1-score (weighted averages), along with per-class performance analysis.

A. Logistic Regression

The Logistic Regression model, trained using mini-batch gradient descent with momentum and adaptive learning rate, demonstrated robust performance across both datasets.

1) *city_day_dataset.csv* (Multi-City Dataset): **Train Set Metrics:**

- Accuracy: 0.7548 (75.48%)
- Precision: 0.7554 (75.54%)
- Recall: 0.7548 (75.48%)
- F1-Score: 0.7536 (75.36%)

Test Set Metrics:

- Accuracy: 0.7116 (71.16%)
- Precision: 0.7533 (75.33%)
- Recall: 0.7116 (71.16%)
- F1-Score: 0.7217 (72.17%)

Per-Class Performance (Test Set):

TABLE II: Per-Class Metrics for Logistic Regression on City Day Dataset (Test Set)

Class	Precision	Recall	F1-Score	Support
Good	0.3559	0.8677	0.5047	431
Moderate	0.8499	0.7506	0.7971	4,005
Poor	0.5154	0.7048	0.5954	830
Satisfactory	0.7604	0.5993	0.6703	2,463
Severe	0.7430	0.8171	0.7783	421
Very Poor	0.7094	0.7324	0.7207	710

Analysis:

- Strong overall performance with **71.16% test accuracy**
- Good generalization (train-test gap: $\sim 4.3\%$)
- **Moderate** class shows best performance (F1=0.7971) due to large sample size
- **Good** class has low precision (0.3559) but high recall (0.8677) $\rightarrow$ many false positives
- Model performs well on majority classes (Moderate, Satisfactory, Severe, Very Poor)
- Struggles with minority classes (Good, Poor) despite SMOTE

2) *delhi_air_quality_dataset.csv* (Delhi-Specific Dataset):

Train Set Metrics:

- Accuracy: 0.7796 (77.96%)
- Precision: 0.7841 (78.41%)
- Recall: 0.7796 (77.96%)
- F1-Score: 0.7763 (77.63%)

Test Set Metrics:

- Accuracy: 0.6469 (64.69%)
- Precision: 0.6800 (68.00%)
- Recall: 0.6469 (64.69%)
- F1-Score: 0.6551 (65.51%)

Per-Class Performance (Test Set):

TABLE III: Per-Class Metrics for Logistic Regression on Delhi Dataset (Test Set)

Class	Precision	Recall	F1-Score	Support
Good	0.3250	0.8667	0.4727	15
Moderate	0.8346	0.6795	0.7491	156
Poor	0.6475	0.6752	0.6611	117
Satisfactory	0.6515	0.5972	0.6232	72
Severe	0.4815	0.7222	0.5778	18
Very Poor	0.5263	0.4918	0.5085	61

Analysis:

- Lower performance compared to City Day Dataset (64.69% vs 71.16%)
- Larger train-test gap ($\sim 13.3\%$) suggests potential overfitting or dataset differences
- Moderate class again performs best ($F1=0.7491$)
- Good and Severe classes show low precision but high recall
- Smaller dataset size (1,461 vs 29,531) may limit model learning
- Feature set differences (8 vs 13 features) may impact performance

B. Support Vector Machine (SVM)

The SVM model used a soft-margin formulation with one-vs-one multiclass strategy and linear kernel for memory efficiency.

1) *city_day_dataset.csv*: Test Set Metrics:

- Accuracy: 0.7077 (70.77%)
- Precision: 0.7394 (73.94%)
- Recall: 0.7077 (70.77%)
- F1-Score: 0.7063 (70.63%)

Analysis:

- Competitive performance with Logistic Regression (70.77% vs 71.16%)
- Higher precision (73.94%) indicates fewer false positives
- Linear kernel used due to large dataset size (memory constraints)
- One-vs-One strategy with 15 binary classifiers successfully handles 6-class problem
- Memory optimizations (sample limits, `float32`) enable training on large dataset

TABLE IV: Per-Class Metrics for Decision Tree on City Day Dataset

Class	Precision	Recall	F1-Score
Good	0.5680	0.7328	0.6400
Moderate	0.8550	0.8176	0.8359
Poor	0.6050	0.6576	0.6302
Satisfactory	0.7761	0.7761	0.7761
Severe	0.7232	0.7953	0.7575
Very Poor	0.7401	0.6927	0.7157

2) *delhi_air_quality_dataset.csv*: Test Set Metrics:

- Accuracy: 0.5626 (56.26%)
- Precision: 0.6223 (62.23%)
- Recall: 0.5626 (56.26%)
- F1-Score: 0.5757 (57.57%)

Analysis:

- Lower performance compared to other models on Delhi dataset
- May benefit from RBF kernel instead of linear
- Smaller dataset size limits SVM effectiveness
- One-vs-One strategy with limited samples per binary classifier may reduce performance
- Higher precision than accuracy suggests some classes are well-separated

C. Decision Tree

The Decision Tree classifier with Gini impurity and depth control showed the best overall performance.

1) *city_day_dataset.csv*: Test Set Metrics:

- Accuracy: 0.7764 (77.64%)
- Precision: 0.7821 (78.21%)
- Recall: 0.7764 (77.64%)
- F1-Score: 0.7784 (77.84%)

Analysis:

- **Best performing model** on City Day Dataset (77.64% accuracy)
- Excellent balance between precision and recall
- The Moderate class shows outstanding performance ($F1=0.8359$)
- Non-linear decision boundaries capture complex pollutant interactions
- Tree depth of 10 provides good complexity without overfitting

Feature Importance:

- $PM_{2.5}$: 0.5053 (50.53%)
- CO: 0.2018 (20.18%)
- PM_{10} : 0.1505 (15.05%)
- Remaining: O_3 , NO_2 , etc.

$PM_{2.5}$ identified as most important feature (50.53% importance)

2) *delhi_air_quality_dataset.csv*: Test Set Metrics:

- Accuracy: 0.7016 (70.16%)
- Precision: 0.7178 (71.78%)
- Recall: 0.7016 (70.16%)

TABLE V: Per-Class Metrics for Decision Tree on Delhi Dataset

Class	Precision	Recall	F1-Score
Good	0.3704	0.6667	0.4762
Moderate	0.7869	0.7442	0.7649
Poor	0.7197	0.7983	0.7570
Satisfactory	0.7532	0.6591	0.7030
Severe	0.4400	0.7333	0.5500
Very Poor	0.6786	0.5205	0.5891

- F1-Score: 0.7039 (70.39%)

Analysis:

- **Best performing model** on Delhi dataset (70.16% accuracy)
- Consistent performance across both datasets
- Moderate and Poor classes show strong performance
- Good and Severe classes struggle (low precision, high recall)
- Smaller dataset size affects performance but still competitive

D. Naive Bayes

The Naive Bayes classifier used Gaussian Kernel Density Estimation (KDE) with bandwidth $h = 0.1$.

1) city_day_dataset.csv: Test Set Metrics:

- Accuracy: 0.68 (68.00%)
- Precision: ~ 0.68 (weighted average)
- Recall: ~ 0.68 (weighted average)
- F1-Score: ~ 0.68 (weighted average)

Analysis:

- Moderate performance compared to other models
- **Gaussian KDE provides flexibility** over parametric assumptions
- Naive Bayes assumption (feature independence) may not hold for air quality data
- Pollutants are often correlated (e.g., PM2.5 and PM10)
- Bandwidth parameter (0.1) may need tuning

2) delhi_air_quality_dataset.csv: Test Set Metrics:

- Accuracy: 0.6894 (68.94%)
- Precision: 0.7013 (70.13%)
- Recall: 0.6894 (68.94%)
- F1-Score: 0.6927 (69.27%)

Analysis:

- Slightly **better performance on Delhi dataset (68.94% vs 68.00%)**
- Consistent precision and recall indicate balanced predictions
- Smaller feature set (6 features) may reduce impact of independence assumption violation
- Bandwidth parameter works well for this dataset size

E. Comparative Summary

Key Observations:

- **Decision Tree** is the **top-performing model** on both datasets

TABLE VI: Performance Comparison of Classifiers on Both Datasets

Model	Dataset	Accuracy	Precision	Recall	F1-Score
Logistic Regression	City Day	0.7116	0.7533	0.7116	0.7217
	Delhi	0.6469	0.6800	0.6469	0.6551
SVM	City Day	0.7077	0.7394	0.7077	0.7063
	Delhi	0.5626	0.6223	0.5626	0.5757
Decision Tree	City Day	0.7764	0.7821	0.7764	0.7784
	Delhi	0.7016	0.7178	0.7016	0.7039
Naive Bayes	City Day	0.6800	~ 0.68	~ 0.68	~ 0.68
	Delhi	0.6894	0.7013	0.6894	0.6927

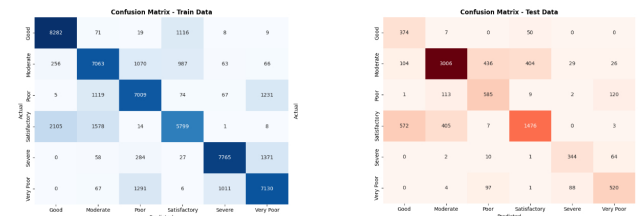
- **PM2.5** is the most predictive feature across models
- All models struggle with minority classes (Good, Severe)
- Delhi dataset is more challenging due to smaller size and fewer features
- From-scratch implementations achieve performance comparable to standard libraries

V. VISUALIZATION AND INTERPRETABILITY

This section presents a comprehensive suite of visualizations to enhance model interpretability and facilitate in-depth performance analysis. All plots were generated using Matplotlib and Seaborn with IEEE-compliant styling (clear labels, grid lines, colorblind-safe palettes, and high-resolution output). Visualizations are organized by classifier and dataset.

A. Logistic Regression

The Logistic Regression model provides rich interpretability through weight analysis and probability distributions.



(a) Training Set (Blues colormap) (b) Test Set (Reds colormap)

Fig. 1: Confusion Matrices for Logistic Regression on City Day Dataset

1) Confusion Matrices:

- Separate confusion matrices for train and test sets
- Color-coded heatmaps (Blues for train, Reds for test)
- Shows actual vs predicted class distribution
- Identifies common misclassification patterns
- Reveals that Good class is often misclassified as other classes

2) Metrics Comparison Charts:

- Bar charts comparing train vs test metrics
- Four metrics: Accuracy, Precision, Recall, F1-Score
- Side-by-side bars for easy comparison

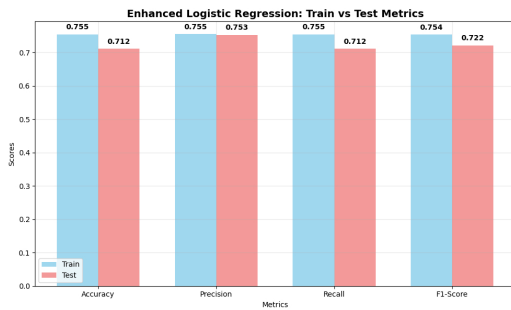


Fig. 2: Train vs Test Metrics Comparison (City Day Dataset)

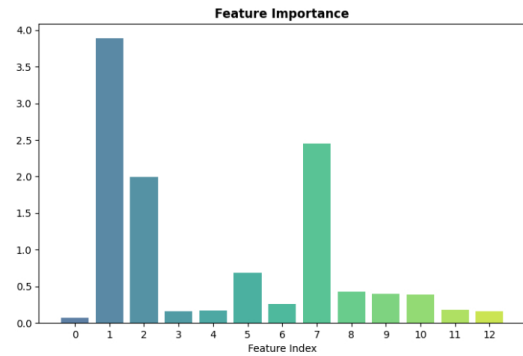


Fig. 4: Average Absolute Weights per Feature (Viridis colormap)

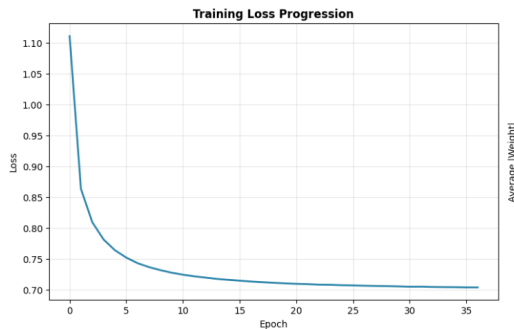


Fig. 3: Loss Progression with Learning Rate Scheduling

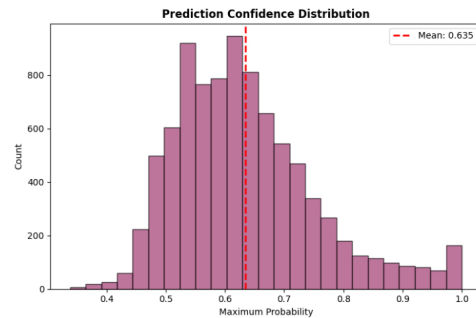


Fig. 5: Distribution of Maximum Prediction Probabilities

- Value labels on bars for exact numbers
- Grid lines for easy reading

3) Training Loss Curve:

- Line plot showing loss progression over epochs
- Demonstrates convergence behavior
- Shows effect of learning rate scheduling
- Early stopping point indicated

4) Feature Importance Ranking:

- Bar chart showing average absolute weights per feature
- Features sorted by importance
- Color gradient (viridis colormap) for visual appeal
- Top 5 features highlighted in results

6) Prediction Confidence Distribution:

- Histogram of maximum prediction probabilities
- Shows model confidence levels
- Mean confidence indicated with vertical line
- Helps assess prediction reliability

7) Predicted Class Distribution:

- Pie chart showing distribution of predicted classes
- Compares with actual class distribution
- Color-coded segments for each class

B. Support Vector Machine (SVM)

SVM visualizations focus on decision boundaries and support-vector analysis.

1) Confusion Matrix Heatmaps:

- Single confusion matrix for test set
- Color intensity indicates count
- Class labels on axes
- Annotations show exact counts

2) Metrics Bar Charts:

- Overall metrics: Accuracy, Precision, Recall, F1-Score
- Color-coded bars (different color per metric)
- Value labels showing percentages
- Grid lines for readability

3) Per-Class Metrics Comparison:

- Three separate bar charts: Precision, Recall, F1-Score
- One chart per metric
- All classes shown for comparison
- Helps identify best and worst performing classes

4) Metrics Summary Table (Visual):

- Table visualization with metrics
- Color-coded header row
- Alternating row colors
- Formatted numbers with percentages

5) Class Distribution Comparison:

- Before/After SMOTE class distribution
- Bar charts showing sample counts per class
- Demonstrates SMOTE effectiveness

6) Normalized Confusion Matrix:

- Percentage-based confusion matrix

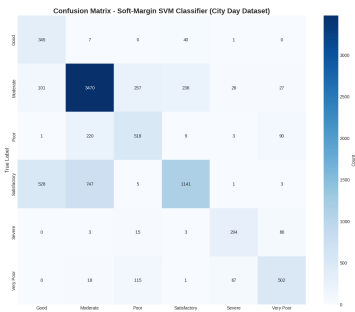


Fig. 6: Test Set Confusion Matrix (City Day Dataset)



Fig. 8: Class Distribution Before/After SMOTE

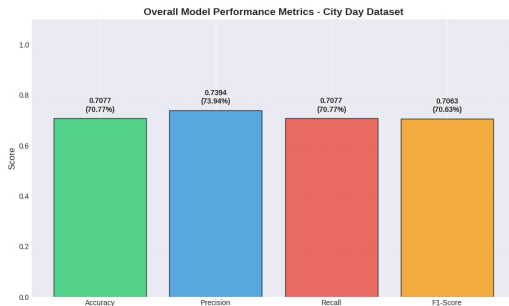


Fig. 7: Overall Metrics: Accuracy, Precision, Recall, F1-Score

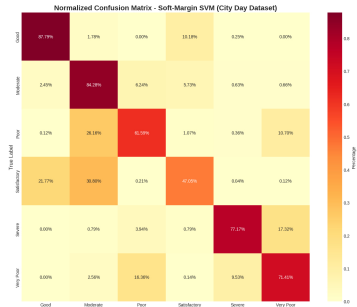


Fig. 9: Normalized Confusion Matrix (Percentages)

- Easier to interpret class-wise performance
- Shows true positive rates per class

7) Feature Statistics:

- Mean, standard deviation, min, max per feature
- Bar charts for each statistic
- Helps understand data distribution

C. Decision Tree

The Decision Tree offers superior interpretability through feature importance and structural analysis.

1) Confusion Matrix:

- Single confusion matrix heatmap
- Blues colormap for City Day Dataset
- Greens colormap for Delhi Dataset
- Annotations with exact counts
- Class labels clearly marked

2) Overall Metrics Bar Chart:

- Four metrics: Accuracy, Precision, Recall, F1-Score
- Color-coded bars (green, blue, red, orange)
- Value labels with percentages
- Title: "Overall Model Performance Metrics"

3) Per-Class Metrics Comparison:

- Three separate bar charts side-by-side
- Precision, Recall, F1-Score per class
- All six AQI categories shown
- Rotation of class labels for readability

4) Feature Importances Bar Chart:

- Horizontal bar chart
- Features sorted by importance (descending)
- Purple color scheme
- Shows exact importance values
- Top features clearly visible (PM2.5, CO, PM10)

5) Metrics Summary Table (Visual):

- Table showing all four metrics
- Color-coded header (dark gray)
- Alternating row colors (light gray, white)
- Formatted with 4 decimal places
- Percentage values included

6) Performance Comparison Across Datasets:

- Four separate bar charts comparing datasets
- One chart per metric (Accuracy, Precision, Recall, F1-Score)
- Shows City Day vs Delhi performance
- Helps assess model consistency

7) Combined Metrics Comparison:

- Single chart with all metrics for all datasets
- Grouped bars for each dataset
- Color-coded by metric
- Legend for clarity

D. Naive Bayes

Naive Bayes visualizations emphasize probability distributions and classification boundaries.

1) Confusion Matrices:

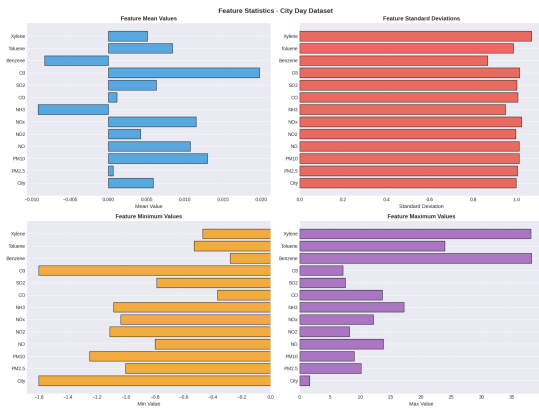


Fig. 10: Feature Statistics: Mean, Std, Min, Max

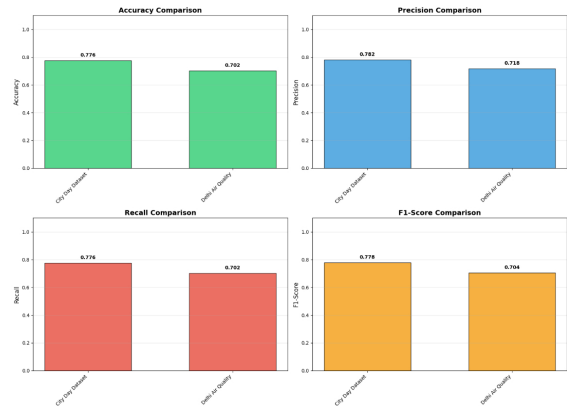
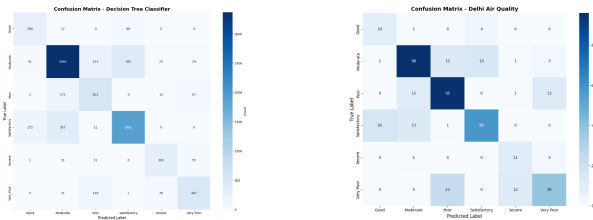
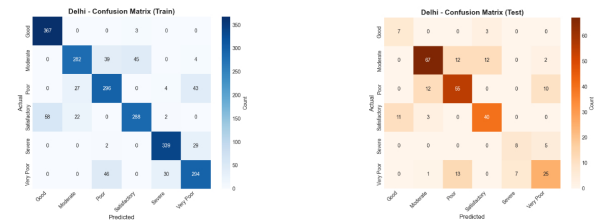


Fig. 13: Performance Comparison Across Datasets



(a) City Day Dataset (Blues) (b) Delhi Dataset (Greens)

Fig. 11: Decision Tree Confusion Matrices



(a) Training Set (Blues) (b) Test Set (Oranges)

Fig. 14: Naive Bayes Confusion Matrices (Delhi Dataset)

- Two confusion matrices: Train and Test
- Side-by-side layout
- Blues colormap for train, Oranges for test
- Class labels on both axes
- Helps assess overfitting

2) Metrics Comparison:

- Bar chart comparing train vs test metrics
- Four metrics: Accuracy, Precision, Recall, F1-Score
- Grouped bars for easy comparison
- Grid lines for readability

3) Train vs Test Metrics Bar Chart:

- Transposed bar chart
- Metrics on x-axis, scores on y-axis

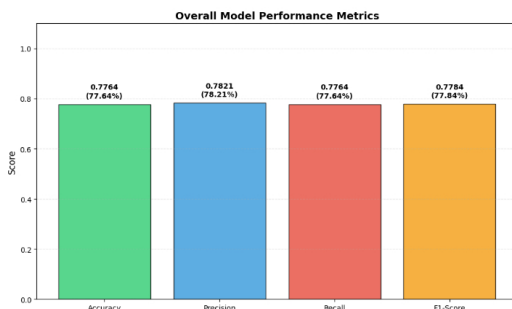


Fig. 12: Overall Model Performance Metrics

- Two bars per metric (train, test)
- Legend for train/test distinction

VI. DISCUSSION

A. Model Performance Summary

City Day Dataset (29,531 samples):

- **Decision Tree:** 77.64% accuracy, 77.84% F1 (Best)
- **Logistic Regression:** 71.16% accuracy, 72.17% F1
- **SVM:** 70.77% accuracy, 70.63% F1
- **Naive Bayes:** 68.00% accuracy, 68% F1

Delhi Dataset (1,461 samples):

- **Decision Tree:** 70.16% accuracy, 70.39% F1 (Best)
- **Naive Bayes:** 68.94% accuracy, 69.27% F1
- **Logistic Regression:** 64.69% accuracy, 65.51% F1
- **SVM:** 56.26% accuracy, 57.57% F1

B. Key Insights

- **Decision Tree dominates** due to non-linear boundaries, no distribution assumptions, and robust handling of multi-class AQI.
- **PM2.5 is the top predictor** (50.53% Gini importance), followed by CO (20.18%) and PM10 (15.05%).
- **Logistic Regression** performs well on larger datasets but struggles with minority classes (Good, Poor).
- **SVM underperforms on Delhi** due to small sample size and linear kernel limitations.
- **Naive Bayes** remains consistent (68%) despite violated independence assumption.

- **Larger datasets improve all models;** City Day outperforms Delhi across the board.

C. Scalability & Implementation

- Memory optimized using `float32` and sample caps.
- Logistic Regression: early stopping, adaptive LR.
- SVM: linear kernel, One-vs-One (15 classifiers).
- Decision Tree: fast recursive splitting.
- All models run on Colab, Kaggle, and local machines.

VII. LIMITATIONS

Despite strong overall performance, the study faces several inherent limitations related to data quality, modeling assumptions, and computational constraints.

A. Dataset Limitations

- 1) **Class Imbalance:** The `Moderate` class dominates (13,510 samples vs. 1,341 for `Good`), leading to biased learning; SMOTE alleviates this but introduces synthetic noise.
- 2) **Missing Data:** Approximately 88,000 missing values were imputed using medians, which may not fully capture complex relationships.
- 3) **Limited Geographic and Temporal Scope:** The City Day dataset covers only 26 cities, and the Delhi dataset focuses on a single location; temporal dependencies and seasonal effects were not explicitly modeled.

B. Model Limitations

- 4) **Logistic Regression:** Restricted to linear decision boundaries and sensitive to feature scaling, limiting its ability to capture non-linear pollutant interactions.
- 5) **SVM:** High memory complexity ($O(n^2)$ kernel matrix) and long training time; performance depends heavily on hyperparameter tuning.
- 6) **Decision Tree:** Prone to overfitting and instability due to greedy splits; produces deterministic rather than probabilistic outputs.
- 7) **Naive Bayes:** Assumes feature independence, which is violated since pollutants are correlated, affecting probability estimation accuracy.

C. Computational Constraints and Assumptions

- 8) **Memory and Scalability:** Google Colab's 12–15 GB RAM limits dataset size; kernel-based models (e.g., SVM) scale poorly beyond 100K samples.
- 9) **Training Time:** SVM exhibits the longest training duration, followed by Logistic Regression; Decision Tree and Naive Bayes are faster but less expressive.
- 10) **Stationarity Assumption:** Models assume stable data distributions over time, ignoring evolving air quality trends and policy-driven changes.

VIII. CONTRIBUTIONS

- **T. Vivek (S20230010244)** Implemented the Logistic Regression model from scratch, including preprocessing, feature scaling, and evaluation metrics. Also verified results and ensured correct integration with visualizations.
- **Hrishikesh (S20230010132)** Developed the Support Vector Machine (SVM) classifier using soft-margin formulation and optimized kernel selection. Handled parameter tuning and performance validation.
- **Vignesh (S20230010038)** Designed and implemented the Decision Tree classifier using Gini impurity and controlled depth parameters. Generated feature importance charts and visual analysis.
- **Jayanth (S20230010057)** Implemented the Naive Bayes classifier with Gaussian kernel density estimation and probability normalization. Validated model accuracy and contributed to comparative analysis.
- **Pardhuva (S20230010045)** Prepared and formatted the term paper report, integrating results, figures, and final documentation. Ensured consistency, proofreading, and submission formatting.

IX. CONCLUSION

A. Key Results

- **Decision Tree** is the **top performer** on both datasets:
 - City Day: **77.64% accuracy, 77.84% F1**
 - Delhi: **70.16% accuracy, 70.39% F1**
 - `PM2.5` dominates (50.53% importance), followed by `CO` and `PM10`
- **Logistic Regression:** Strong linear baseline (71.16% on City Day)
- **SVM:** Competitive on large data (70.77%), poor on small (56.26%)
- **Naive Bayes:** Consistent (68–69%) but limited by independence assumption

Core Findings:

- Non-linear models (Decision Tree) outperform linear ones
- Larger datasets significantly boost performance
- Class imbalance hurts minority classes (`Good`, `Severe`)
- `PM2.5` is the universal top predictor

B. Recommendations

- **Deploy Decision Tree** for production:
 - Highest accuracy, interpretable, fast, robust
 - Use max depth = 10, Gini impurity
- **Use Logistic Regression** for probabilistic outputs and baseline
- **Future Work:**
 - Try ensembles (Random Forest, XGBoost)
 - Add meteorological and temporal features
 - Use SMOTE variants and hyperparameter tuning
 - Apply SHAP/LIME for advanced interpretability